SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

IA en la Nube III: Capacidades y Repaso Unidad 3

Parte 1 de 4 — Modelos de Razonamiento Profundo e Inferencia

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) — Ingeniería de Sistemas

Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

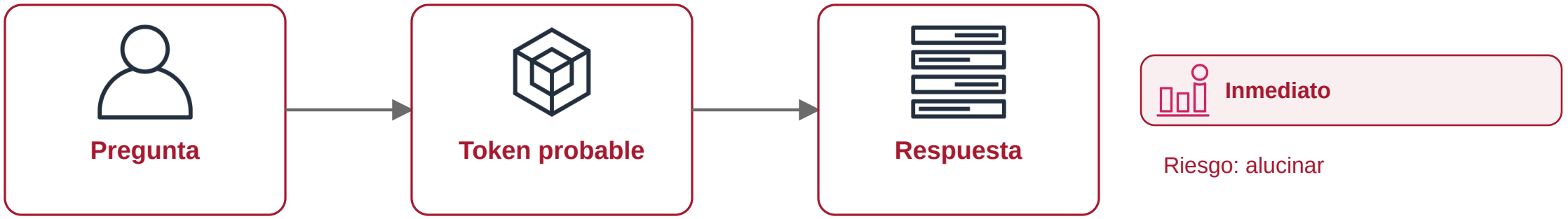
Parte 1 de 4 — Modelos de Razonamiento Profundo e Inferencia

III = tercera (número romano) · IA = inteligencia artificial

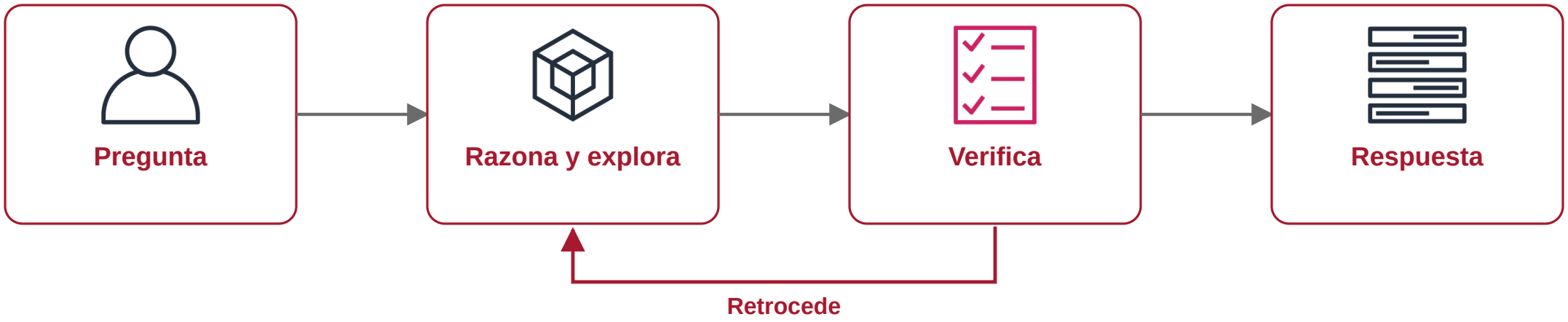
Del reconocimiento de patrones al razonamiento

Respuesta intuitiva frente a deducción paso a paso

Sistema 1 · intuitivo



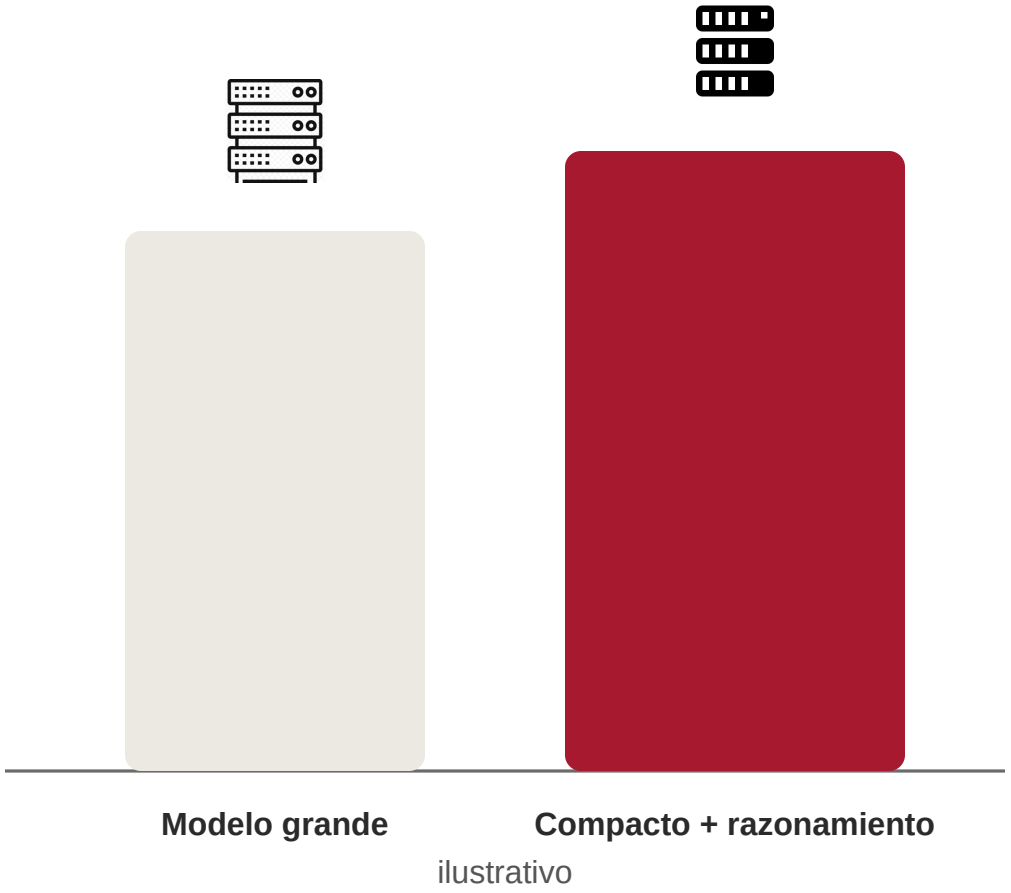
Sistema 2 · deliberativo



Fuente: Kahneman (2011), Pensar rápido y despacio; DeepSeek-AI (2025), arXiv:2501.12948

Escalado del cómputo en tiempo de prueba

Más tokens de razonamiento en la inferencia, sin agrandar el modelo



ilustrativo

Más cómputo al responder

Costo en la API
Se paga por token generado

Menos parámetros
Mismo modelo, más razonamiento

API = interfaz de programación

Fuente: DeepSeek-AI (2025), arXiv:2501.12948; OpenAI, o1 System Card (2024). Barras ilustrativas, sin cifras

Cadena de pensamiento oculta

Bloque de razonamiento no visible, facturado como salida



Se facturan como salida

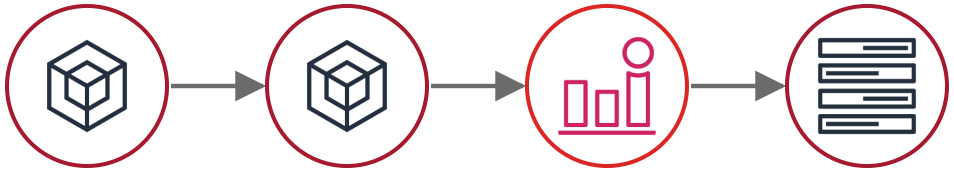
Los tokens think no se muestran, pero cuestan

Fuente: DeepSeek-AI (2025), arXiv:2501.12948 (etiqueta think); tarifas de Amazon Bedrock

Árboles de pensamiento y backtracking

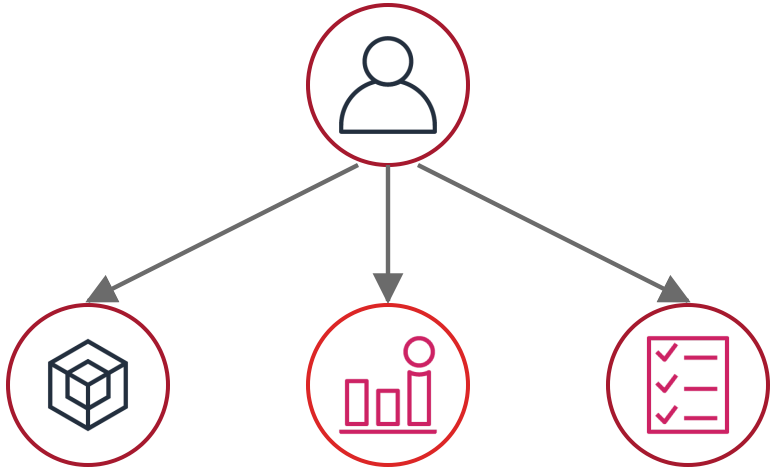
Explorar ramas alternativas y retroceder ante callejones sin salida

Greedy: una sola ruta



Error inicial persiste

Árbol: explora y poda



Rama podada **Camino válido**

Volver atrás evita el efecto bola de nieve

Fuente: Yao et al. (2023), Tree of Thoughts, arXiv:2305.10601

Entrenamiento sin supervisión humana: GRPO

Grupo de respuestas comparadas con recompensas verificables



Sin anotadores humanos en el bucle

GRPO = optimización de política relativa de grupo

Fuente: DeepSeek-AI (2025), arXiv:2501.12948 (GRPO)

Verificación determinista y recompensas

Un compilador o un test juzga mejor que una preferencia humana

Preferencia humana



Anotador



Suena mejor
Reward hacking



Premia lo persuasivo

Verificador determinista



Código
Sandbox y tests



+1 o 0



Premia la corrección

Fuente: DeepSeek-AI (2025), arXiv:2501.12948 (recompensas basadas en reglas)

Anatomía de los tokens de razonamiento

Tokens ocultos y visibles: ambos se facturan como salida



Total = entrada + pensamiento + respuesta

Entrada

Pensamiento (oculto)
Facturado como salida

Respuesta visible
Tarifa de salida

Fuente: tarifas de salida de Amazon Bedrock, archivo local precios-bedrock-us-east-1.txt (consulta 2026-11-05)

Costo por consulta con y sin razonamiento

Con Claude Sonnet 5.5, el razonamiento cuesta unas ocho veces más



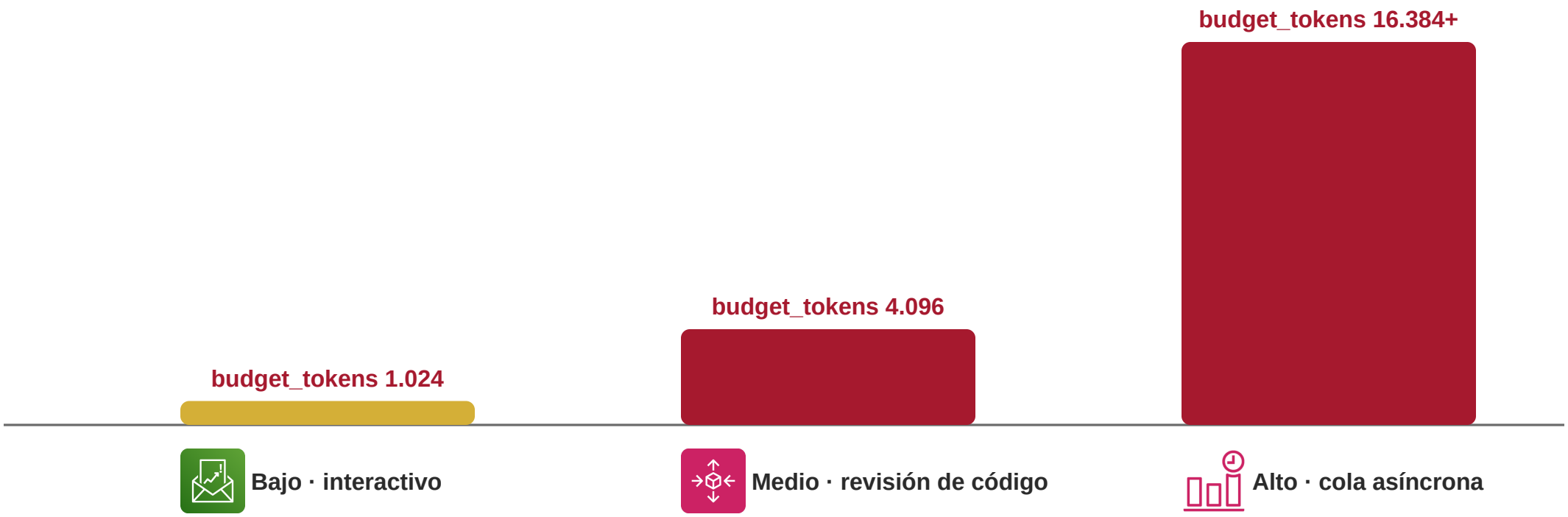
Entrada 1 000; salida 400 o 4 500 tokens

Unas ocho veces más costo

Fuente: cálculo propio con tarifas de Amazon Bedrock us-east-1 ($2 entrada y $10 salida por millón; consulta 2026-11-05)

Control del presupuesto de razonamiento

Parámetros de API para regular la profundidad del pensamiento

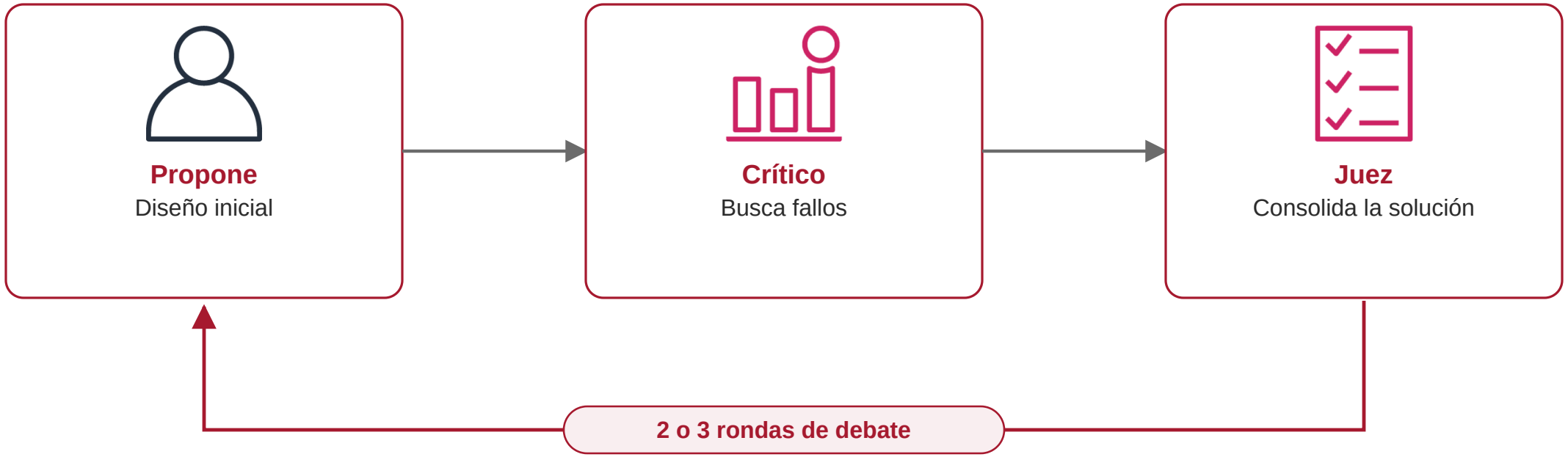


API = interfaz de programación

Fuente: Anthropic, extended thinking (platform.claude.com): budget_tokens mínimo 1.024; deprecado en modelos 4.7 y posteriores

Auto-corrección y debate multi-agente

Un proponente, un crítico y un juez: tres roles para reducir errores

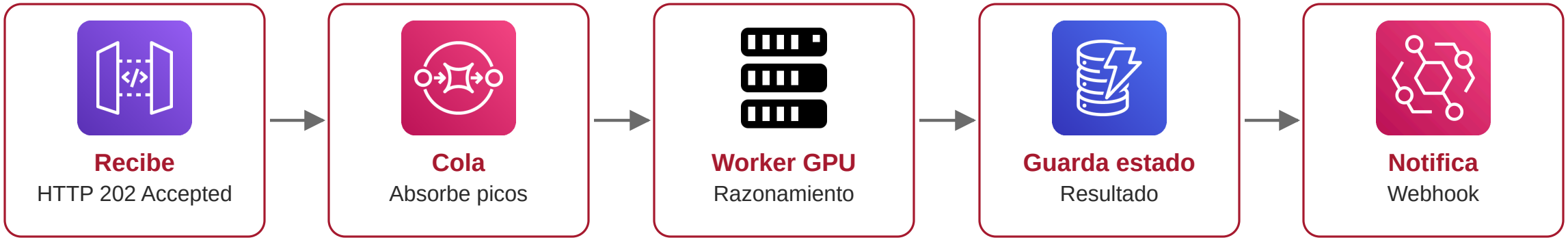


Fuente: Du et al. (2023), Improving Factuality and Reasoning through Multiagent Debate, arXiv:2305.14325

Arquitectura asíncrona para modelos de razonamiento

Cola y notificación para absorber esperas largas





Ninguna conexión espera minutos

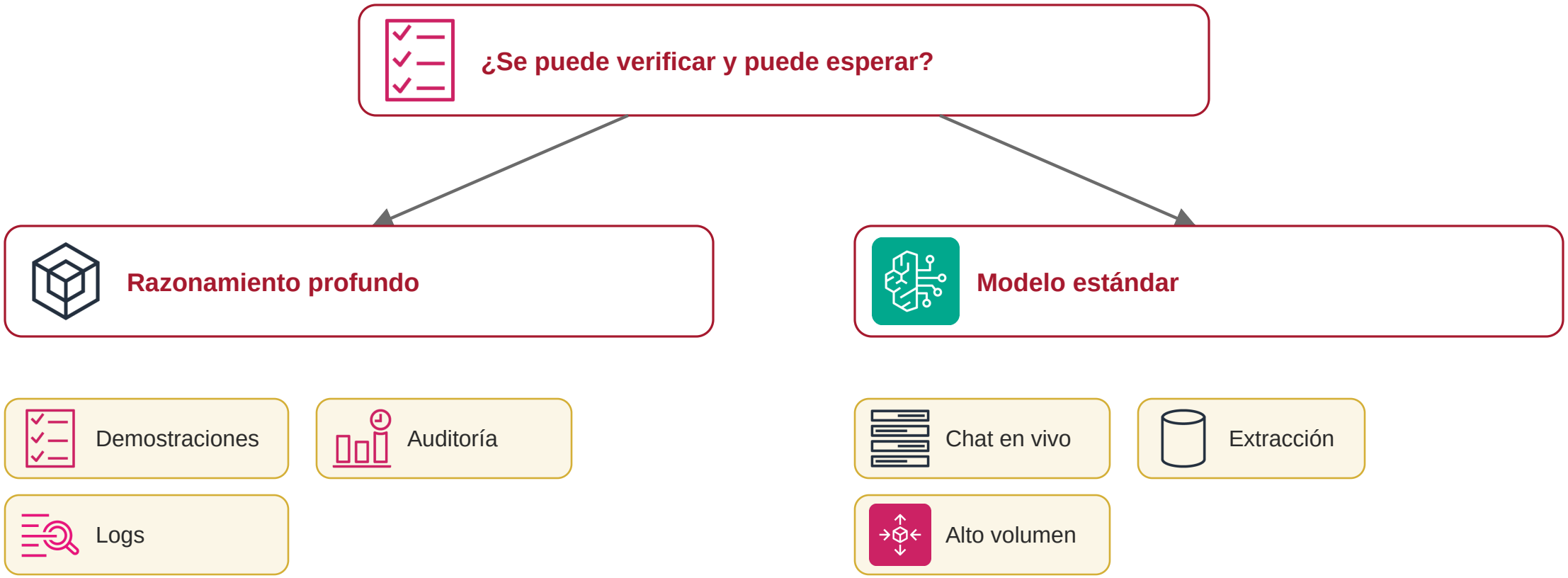
HTTP = protocolo de transferencia de hipertexto · GPU = unidad de procesamiento gráfico

Fuente: arquitectura de referencia con Amazon API Gateway, SQS y EventBridge · Foto: Victorgrigas, Wikimedia Commons (CC BY-SA 3.0)

CRITERIOS DE SELECCIÓN

¿Cuándo usar razonamiento profundo?

Alta complejidad lógica justifica la espera; lo rutinario exige inmediatez



Fuente: criterio de diseño del curso; modelos de Amazon Bedrock (aws.amazon.com/bedrock)

CASO ILUSTRATIVO DE ARQUITECTURA

Diagnóstico en nube privada

Caso ilustrativo, no es una empresa real: fallo de red en OpenStack

LLM convencional

500 MB de logs

Sugiere reiniciar Nova

Modelo de razonamiento

Cruza marcas de tiempo

Causa: split-brain en Galera

Plan en orden

Restaura quórum

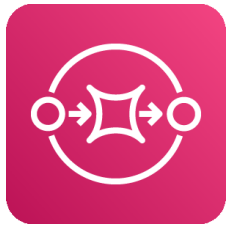
LLM = modelo de lenguaje de gran tamaño · MB = megabyte

Caso ilustrativo de elaboración propia. Galera Cluster (mariadb.com) y OpenStack (docs.openstack.org)

Principios clave de razonamiento en la nube

Cuatro directrices para integrar modelos de razonamiento

1



Asíncrono

Colas y webhooks

2



FinOps

Topes de tokens

3



Verificar

Tests y compiladores

4



Enrutar

Solo consultas complejas

Razonar cuesta: decidir cuándo conviene

Fuente: síntesis de las diapositivas 2 a 14 de esta parte

SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

Programación Asistida y Micro-Sandboxing

Parte 2 de 4 — Generación de Código, Evaluación en SWE-bench y Aislamiento en Nube

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) — Ingeniería de Sistemas

Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

Parte 2 de 4 — Generación de Código, Evaluación en SWE-bench y Aislamiento en Nube

III = tercera (número romano) · SWE = ingeniería de software

Modelos especializados en código

Gestionados en la nube o abiertos para desplegar en casa

Gestionados

- **Claude Sonnet 5.5**
- **Amazon Nova Pro**

Abiertos

- **DeepSeek Coder V2**
- **Qwen 2.5 Coder**

Agente

- Editar varios archivos
- Ejecutar tests
- Generar regresiones

Fuente: Amazon Bedrock (aws.amazon.com/bedrock); documentación oficial de DeepSeek-Coder-V2 y Qwen2.5-Coder

De funciones aisladas a repositorios reales

Un benchmark sirve mientras todavía separa a los modelos

HumanEval
Una función aislada

SWE-bench Verified
Repositorio real con pruebas

Benchmark saturado

Todavía separa a los modelos

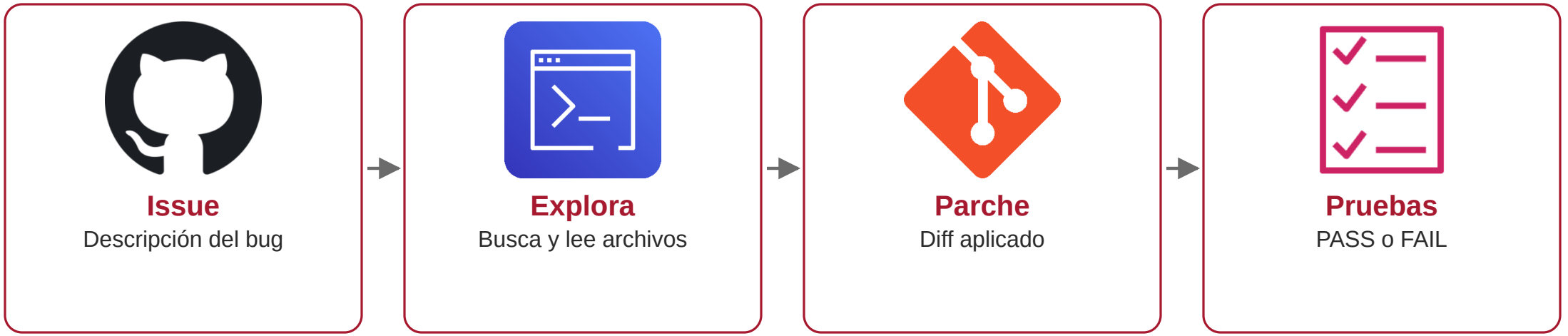
Un benchmark sirve mientras separa modelos

SWE = ingeniería de software

Fuente: Chen et al. (2021), arXiv:2107.03374 (HumanEval); Jimenez et al. (2024), SWE-bench, arXiv:2310.06770

Ciclo de resolución en SWE-bench

Del issue a la validación con pruebas, sin intervención humana



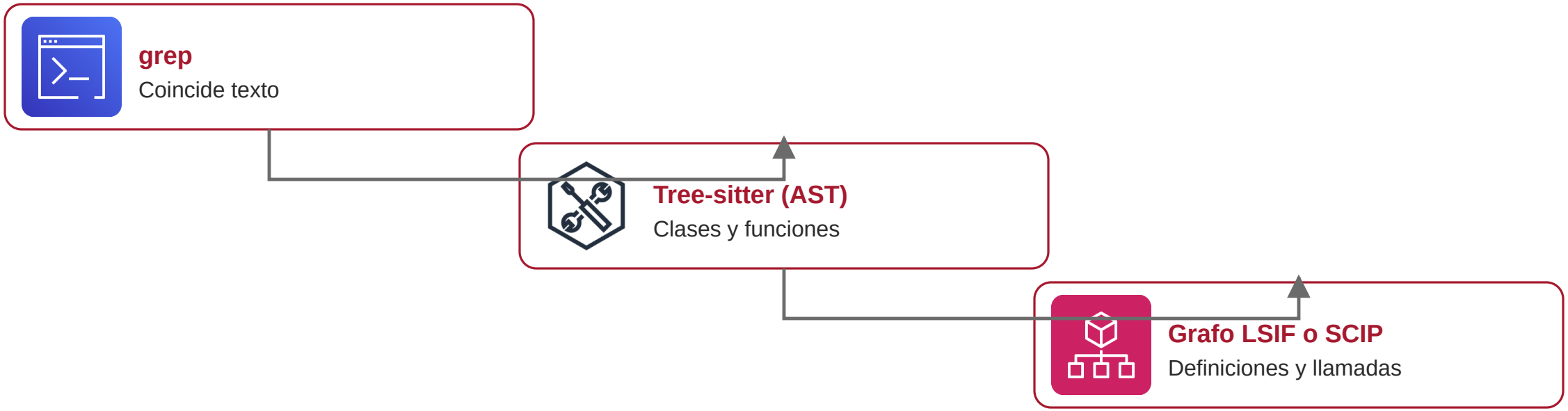
Sin intervención humana en la evaluación

PASS = la prueba se supera · FAIL = la prueba falla · SWE = ingeniería de software

Fuente: swebench.com; Jimenez et al. (2024), arXiv:2310.06770

De grep a árboles sintácticos y grafos

Dar al modelo menos texto, pero el correcto



Menos texto, pero el correcto

LSIF = formato de índice de código · SCIP = protocolo de índice de código

Fuente: tree-sitter.github.io; lsif.dev; github.com/sourcegraph/scip

Edición quirúrgica frente a reescritura completa

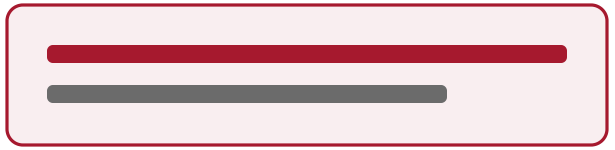
Menos tokens de salida y menos riesgo de truncar el archivo

Reescritura completa



Miles de líneas: riesgo de truncar

Bloque SEARCH / REPLACE



5 a 15 líneas, con coincidencia exacta

El resto del archivo no cambia

SEARCH = búsqueda · REPLACE = reemplazo

Fuente: formato de bloques SEARCH/REPLACE usado en agentes de código (criterio de diseño del curso)

El riesgo de código no confiable

Ejecutar lo que genera un modelo es como ejecutar código de un desconocido



Destrucción

rm -rf · DROP TABLE



Exfiltración

Variables de entorno



Agotamiento

Bucles y memoria

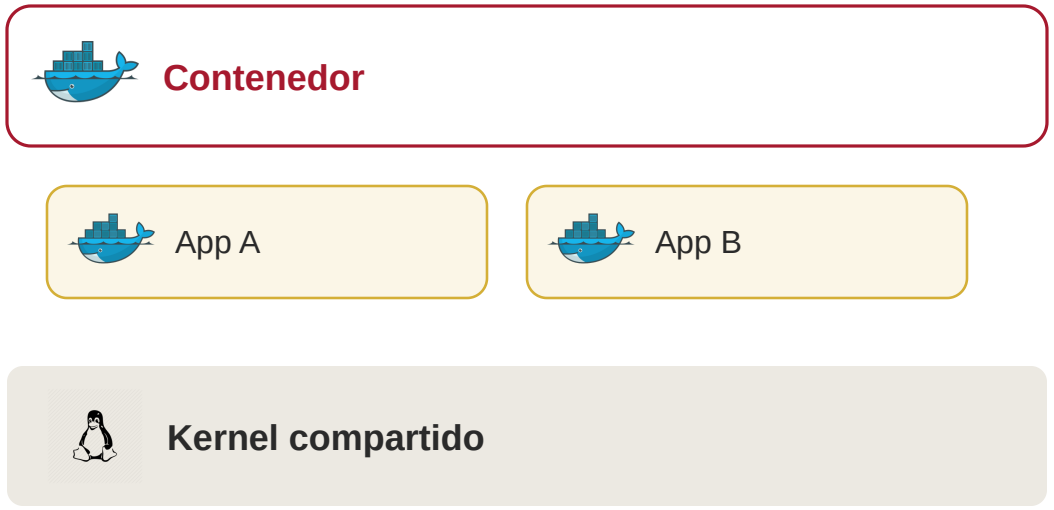
Tratar el código generado como ajeno

TABLE = tabla de base de datos

Fuente: criterio de diseño del curso; OWASP, Top 10 para aplicaciones LLM (owasp.org)

Contenedores frente a microVM

Kernel compartido frente a frontera de hardware



Un fallo del kernel expone todo

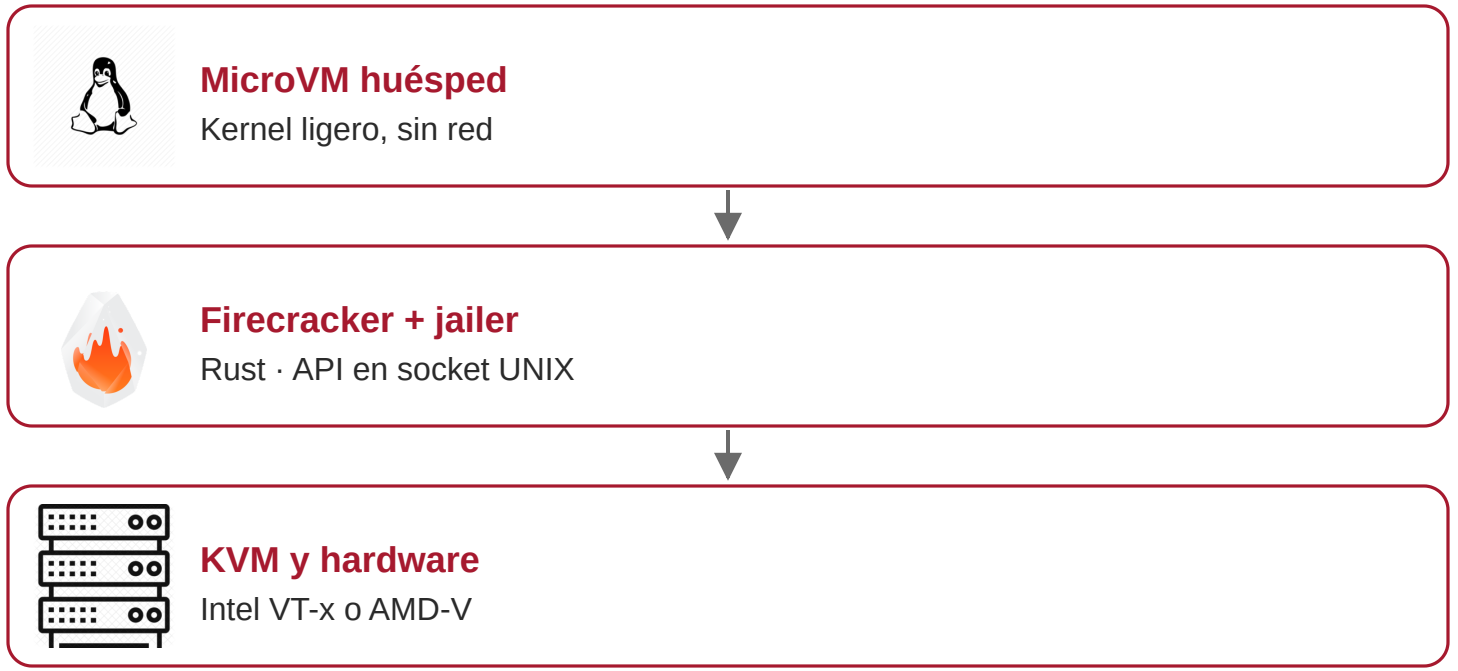


Aislamiento fuerte

Fuente: Linux namespaces y cgroups (kernel.org); Agache et al. (2020), Firecracker, NSDI 2020

Arquitectura de Firecracker

Monitor minimalista escrito en Rust sobre KVM



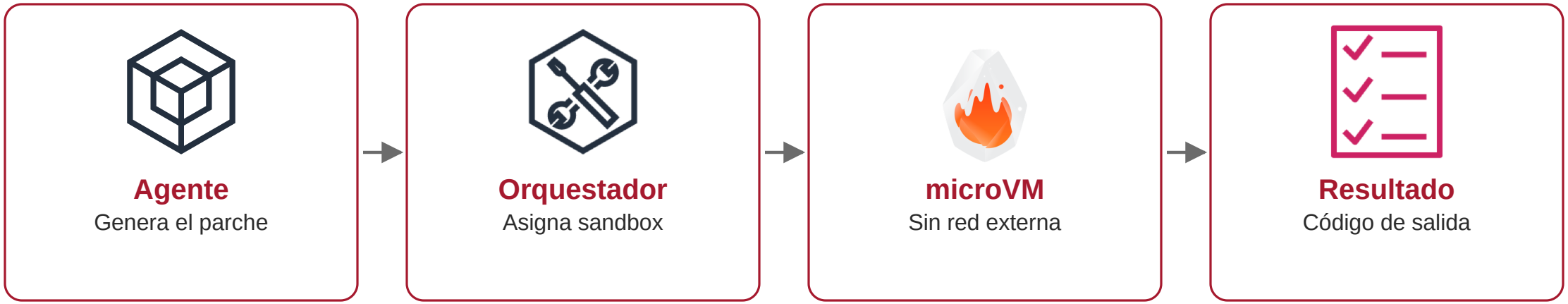
Solo virtio

API = interfaz de programación · UNIX = sistema operativo tipo Unix · KVM = máquina virtual basada en el kernel · VT = tecnología de virtualización · AMD = fabricante de procesadores

Fuente: Agache et al. (2020), Firecracker: Lightweight Virtualization for Serverless Applications, NSDI 2020

Ciclo de ejecución segura en la nube

Agente, orquestador de sandboxes, microVM efímera y destrucción



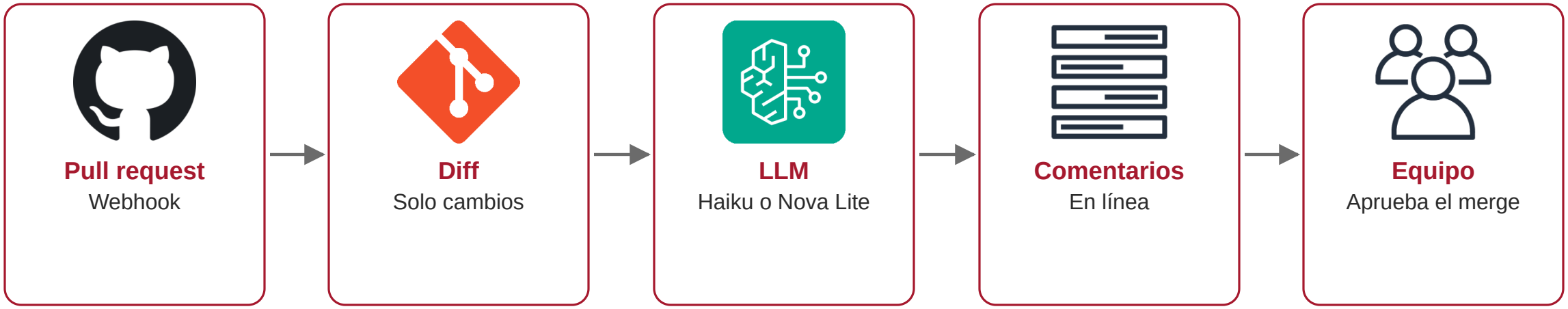
Límite: 30 segundos

Destruir la microVM tras cada prueba

Fuente: patrón de diseño del curso; Firecracker (firecracker-microvm.github.io)

Revisión asistida de pull requests

El modelo comenta; la decisión de aprobar sigue siendo del equipo



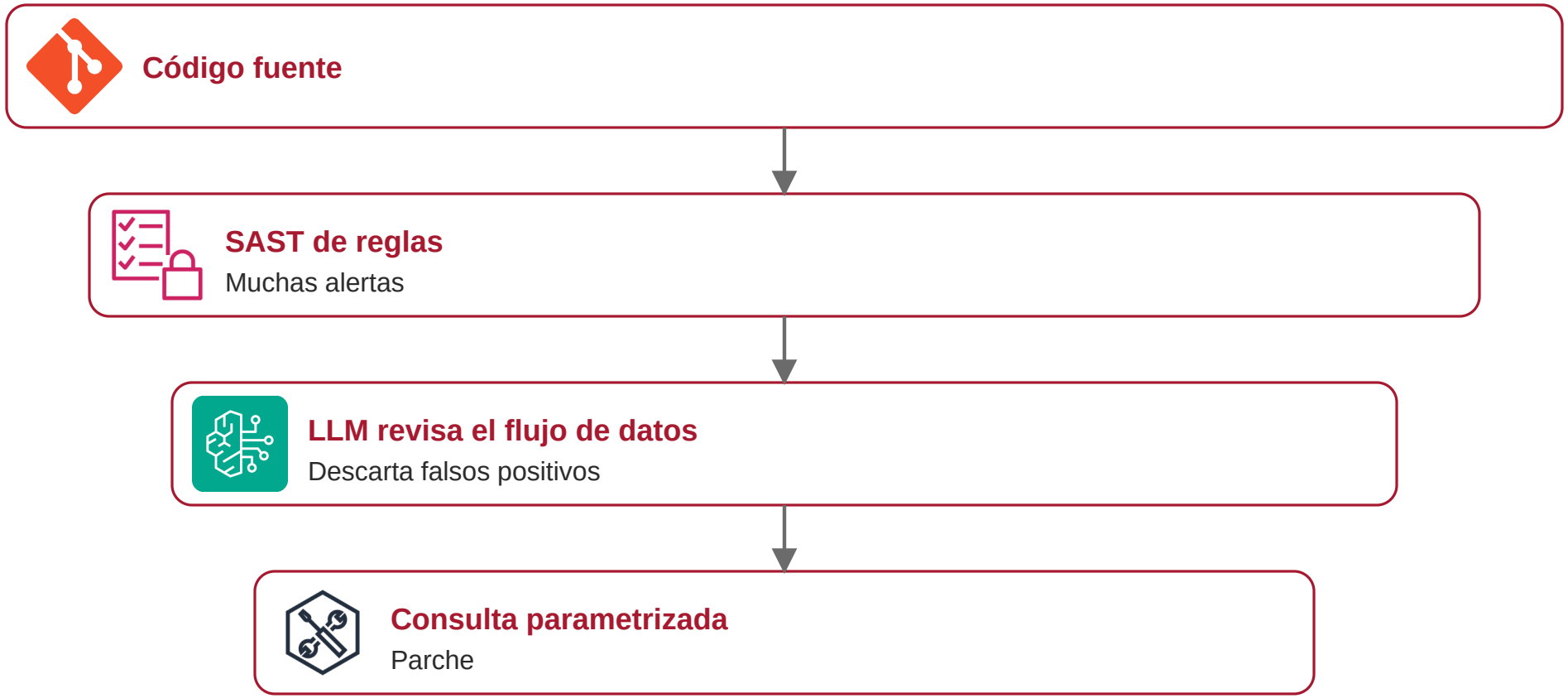
El modelo revisa; el equipo decide

LLM = modelo de lenguaje de gran tamaño

Fuente: GitHub, eventos de pull_request (docs.github.com); Amazon Bedrock

SAST híbrido: reglas deterministas y modelos

Reglas como primer filtro; el modelo descarta falsos positivos

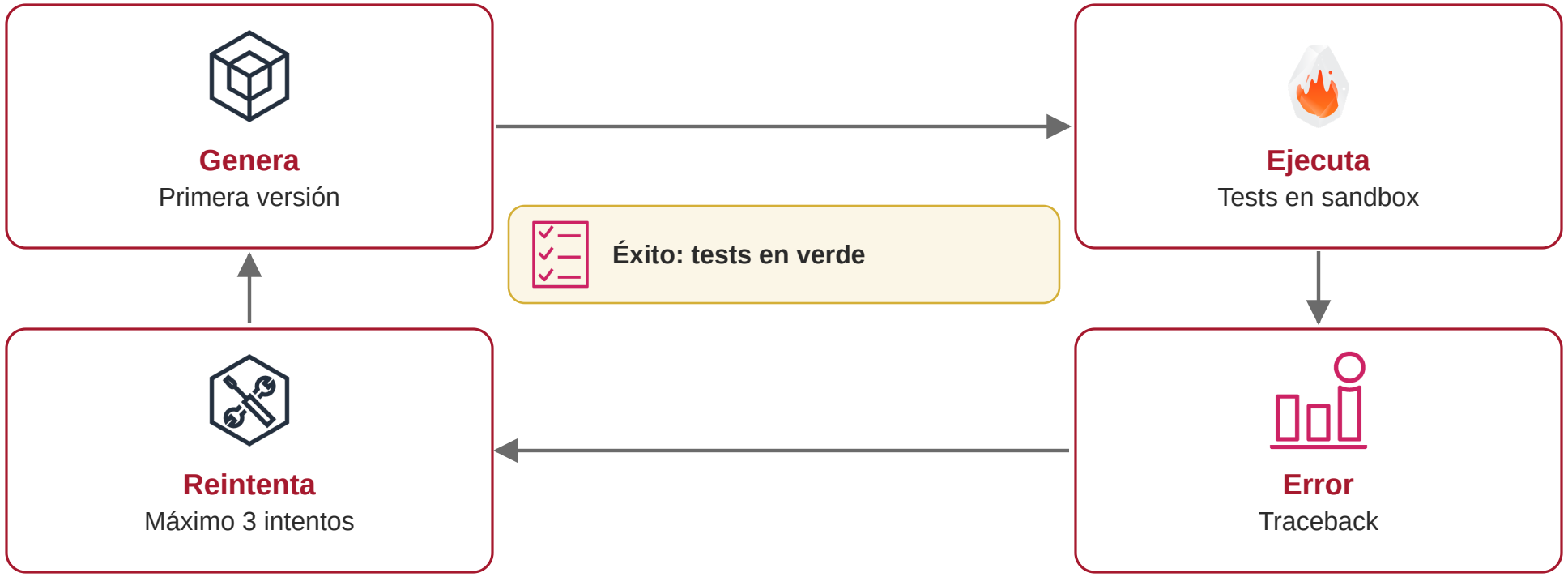


SAST = análisis estático de seguridad · LLM = modelo de lenguaje de gran tamaño

Fuente: patrón de diseño del curso; OWASP Cheat Sheet Series (cheatsheetseries.owasp.org)

Bucle de retroalimentación de error

Compilar, probar, leer el traceback y reintentar, con un máximo de tres



Fuente: patrón de diseño del curso (límite de reintentos configurable)

CASO ILUSTRATIVO

Plataforma de evaluación automatizada de código

Caso ilustrativo, no es una empresa real: el código nunca toca el servidor



El código del estudiante nunca toca el servidor

HMAC = código de autenticación de mensajes con clave · API = interfaz de programación

Caso ilustrativo de elaboración propia; Firecracker y Amazon SQS (documentación de AWS)

Principios de código seguro

Cuatro directrices para integrar modelos de desarrollo

1



Evaluar

SWE-bench, no HumanEval

2



Editar

Bloques SEARCH / REPLACE

3



Aislar

microVM sin red

4



Reintentar

Máximo 3 intentos

SWE = ingeniería de software · SEARCH = búsqueda · REPLACE = reemplazo

Fuente: síntesis de las diapositivas 17 a 29 de esta parte

SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

Sistemas Agénticos y Protocolo MCP

Parte 3 de 4 — Patrón ReAct, Invocación de Herramientas, Memoria y Gobernanza

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) — Ingeniería de Sistemas

Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

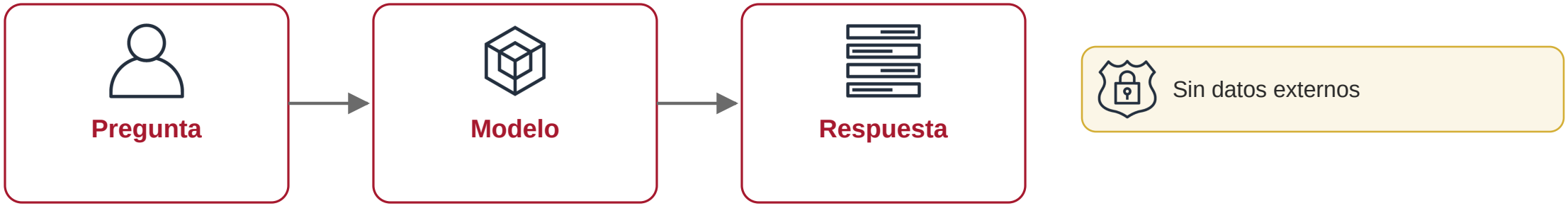
Parte 3 de 4 — Patrón ReAct, Invocación de Herramientas, Memoria y Gobernanza

III = tercera (número romano) · MCP = protocolo de contexto de modelos

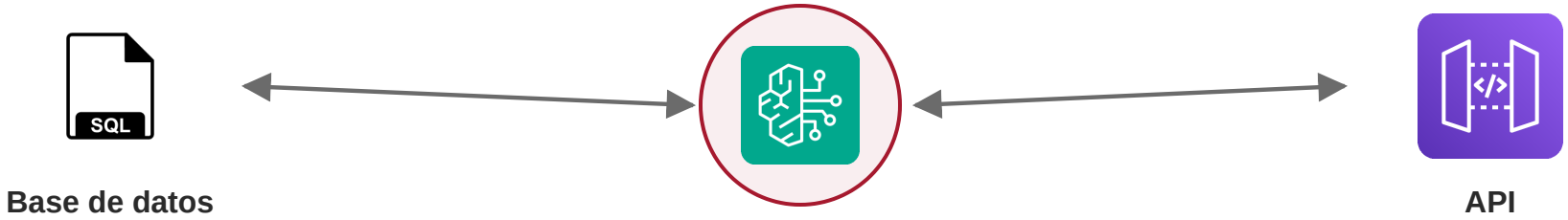
Del chatbot estático al agente autónomo

Un agente usa herramientas, observa resultados y decide el siguiente paso

Chatbot: solo texto



Agente: usa herramientas y observa



API = interfaz de programación

Fuente: patrón de agentes con herramientas (criterio de diseño del curso)

El patrón ReAct: Thought, Action, Observation

Ciclo de razonamiento, llamada a herramienta y observación

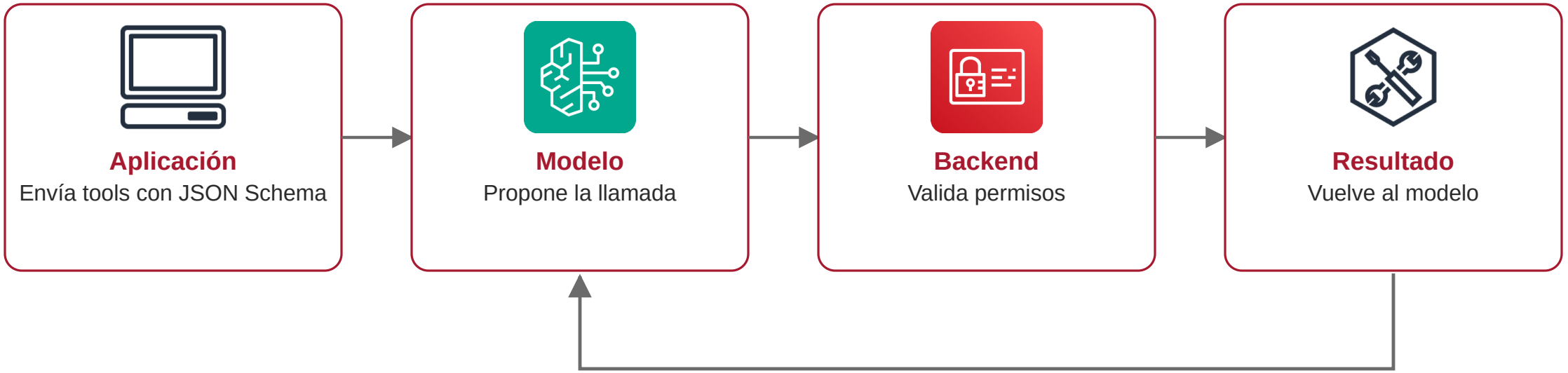


JSON = notación de objetos de JavaScript

Fuente: Yao et al. (2023), ReAct: Synergizing Reasoning and Acting in Language Models, arXiv:2210.03629

Mecanismo técnico de function calling

El modelo propone la llamada; el backend la valida y ejecuta



El modelo no ejecuta nada: propone

JSON = notación de objetos de JavaScript

Fuente: documentación de tool use de Anthropic y JSON Schema (json-schema.org)

Diseño de JSON Schema para herramientas

Nombres de dominio, tipos estrictos y enumeraciones



run_query

query: string

Acepta cualquier texto



consultar_saldo_cuenta

cuenta_id: patrón · tipo: enum

Parámetros validados

Nombres de dominio y tipos estrictos

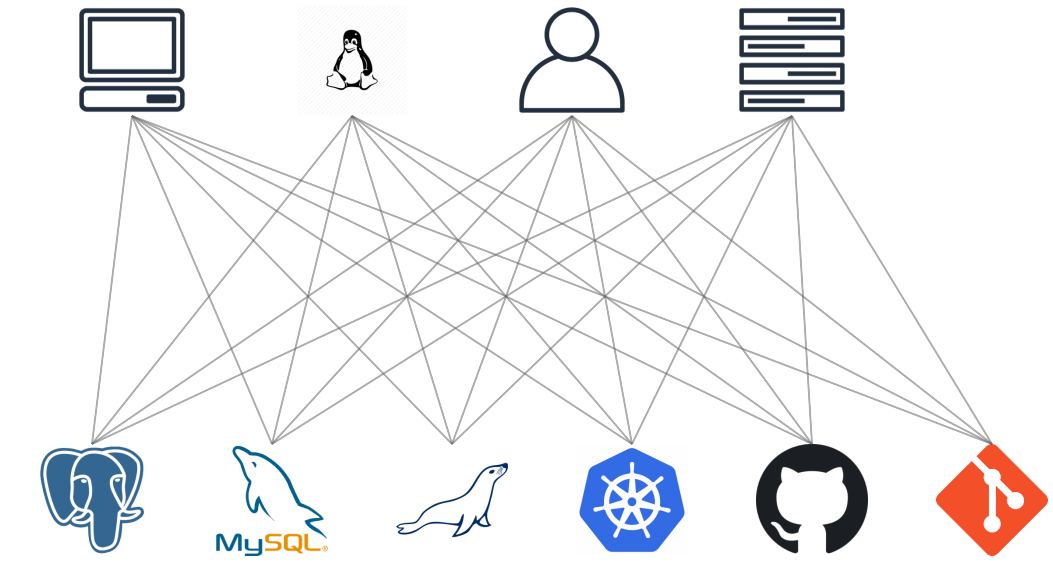
JSON = notación de objetos de JavaScript

Fuente: JSON Schema (json-schema.org); criterio de diseño del curso

Model Context Protocol (MCP): el puerto USB-C de la IA

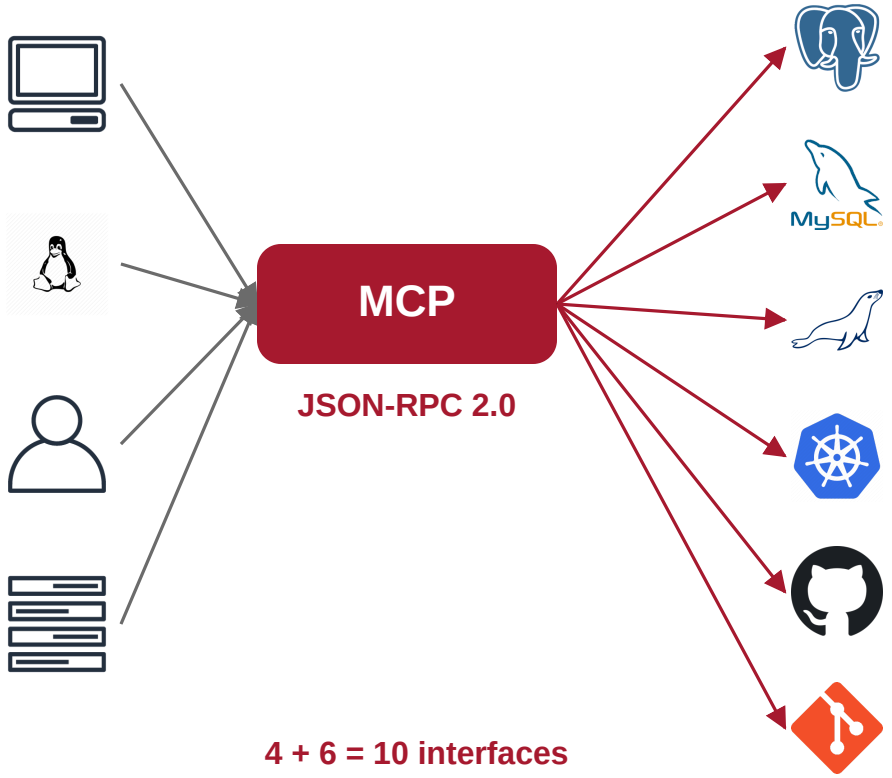
Un protocolo común en lugar de conectores a medida

Sin estándar



4 × 6 = 24 conectores

Con MCP



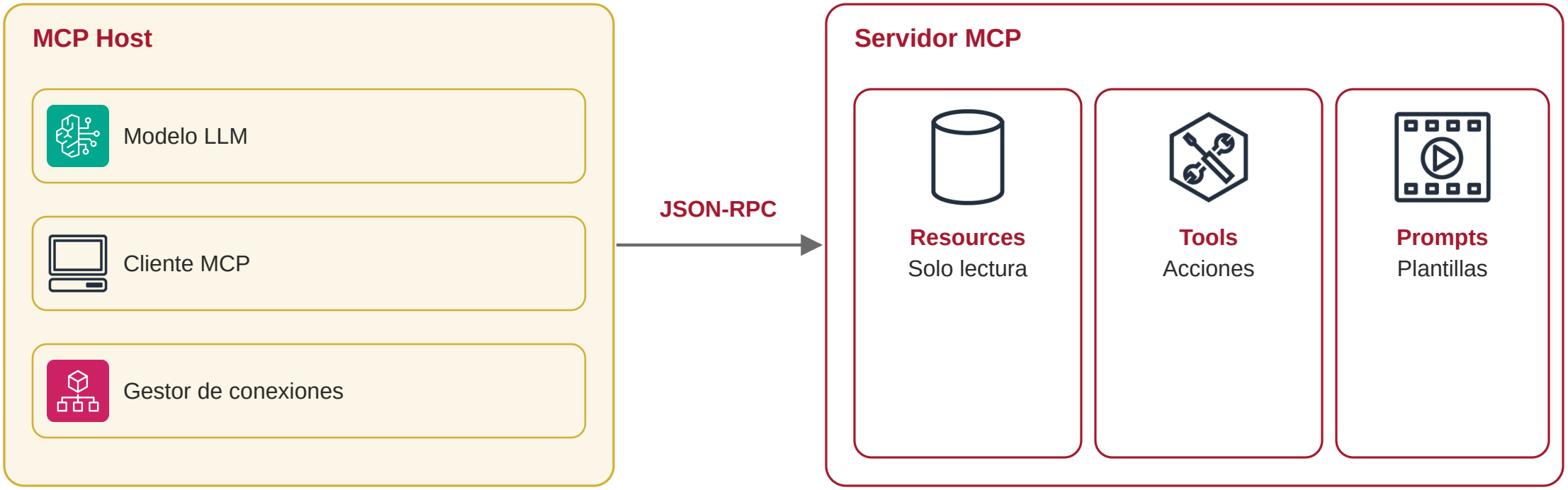
4 + 6 = 10 interfaces

JSON = notación de objetos de JavaScript · RPC = llamada a procedimiento remoto · USB = bus universal en serie · IA = inteligencia artificial

Fuente: Model Context Protocol (modelcontextprotocol.io), especificación oficial; iconos oficiales de cada marca

Arquitectura cliente-host-servidor en MCP

Resources leen, Tools actúan y Prompts guían



MCP = protocolo de contexto de modelos · LLM = modelo de lenguaje de gran tamaño · JSON = notación de objetos de JavaScript · RPC = llamada a procedimiento remoto

Fuente: Model Context Protocol, arquitectura y primitivas (modelcontextprotocol.io)

Transportes MCP: STDIO frente a HTTP remoto

Proceso local sin puertos, o servicio remoto con TLS y OAuth

STDIO: proceso local

Host local

Servidor hijo
stdin / stdout

Sin puertos de red

HTTP remoto

Cliente remoto

Balanceador
OAuth 2.0 y mTLS

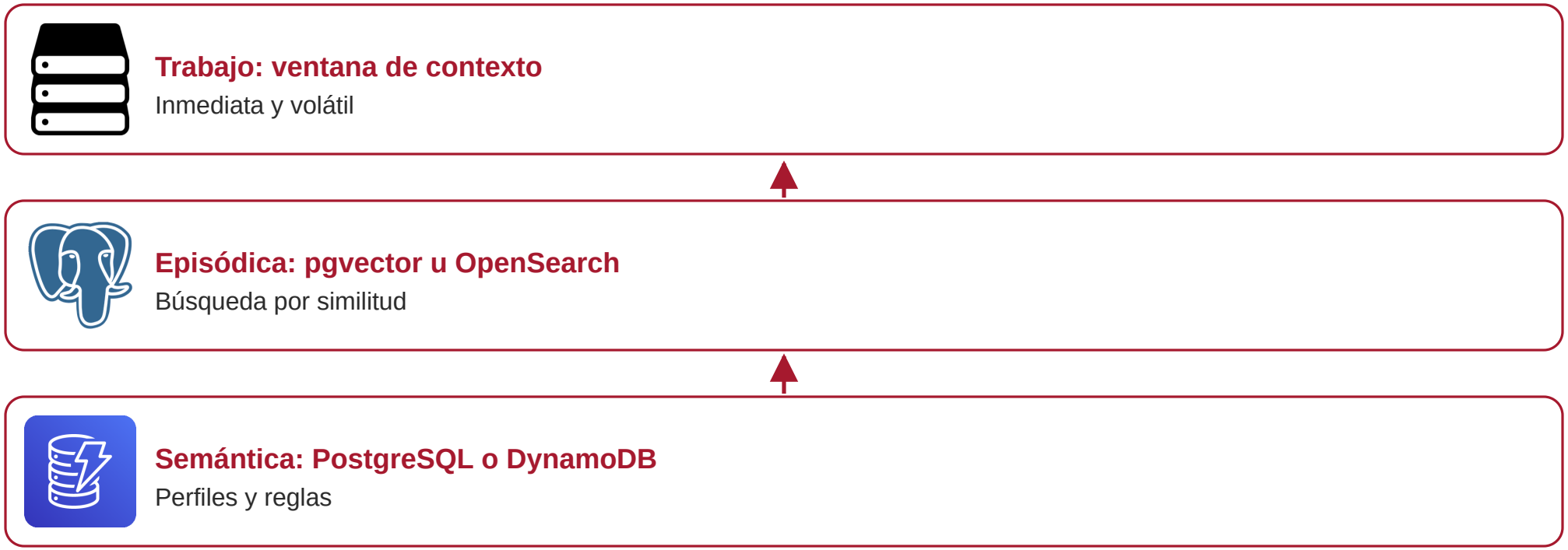
Servidores MCP

STDIO = entrada y salida estándar · HTTP = protocolo de transferencia de hipertexto · MCP = protocolo de contexto de modelos · TLS = seguridad de capa de transporte

Fuente: Model Context Protocol, transportes (modelcontextprotocol.io)

Jerarquía de memoria en sistemas agénticos

Memoria de trabajo, episódica y semántica: RAG recupera lo necesario



RAG recupera lo relevante

RAG = generación aumentada por recuperación

Fuente: pgvector (github.com/pgvector/pgvector); Amazon OpenSearch Service (aws.amazon.com/opensearch-service)

Compactación y ventana deslizante

Dos formas de hacer espacio cuando la ventana se llena

Ventana deslizante



Pierde el prompt de sistema

Compactar



Conserva decisiones, IDs y estado

S = prompt de sistema

Fuente: criterio de diseño del curso (compactación con resumen)

Acotar el radio de impacto (blast radius)

Mínimo privilegio, aprobación humana y cuotas



Mínimo privilegio

Solo lectura, claves cortas



Humano en el bucle

Aprueba lo destructivo



Cuotas

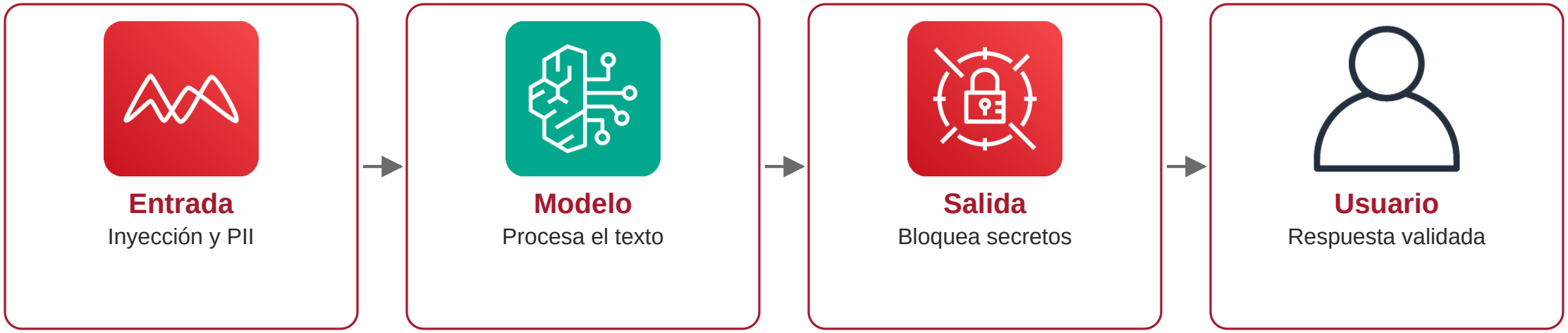
Límite de gasto

El daño queda acotado

Fuente: AWS, mínimo privilegio en IAM (docs.aws.amazon.com/IAM)

Guardrails: filtros de entrada y salida

Capas deterministas y semánticas alrededor del modelo



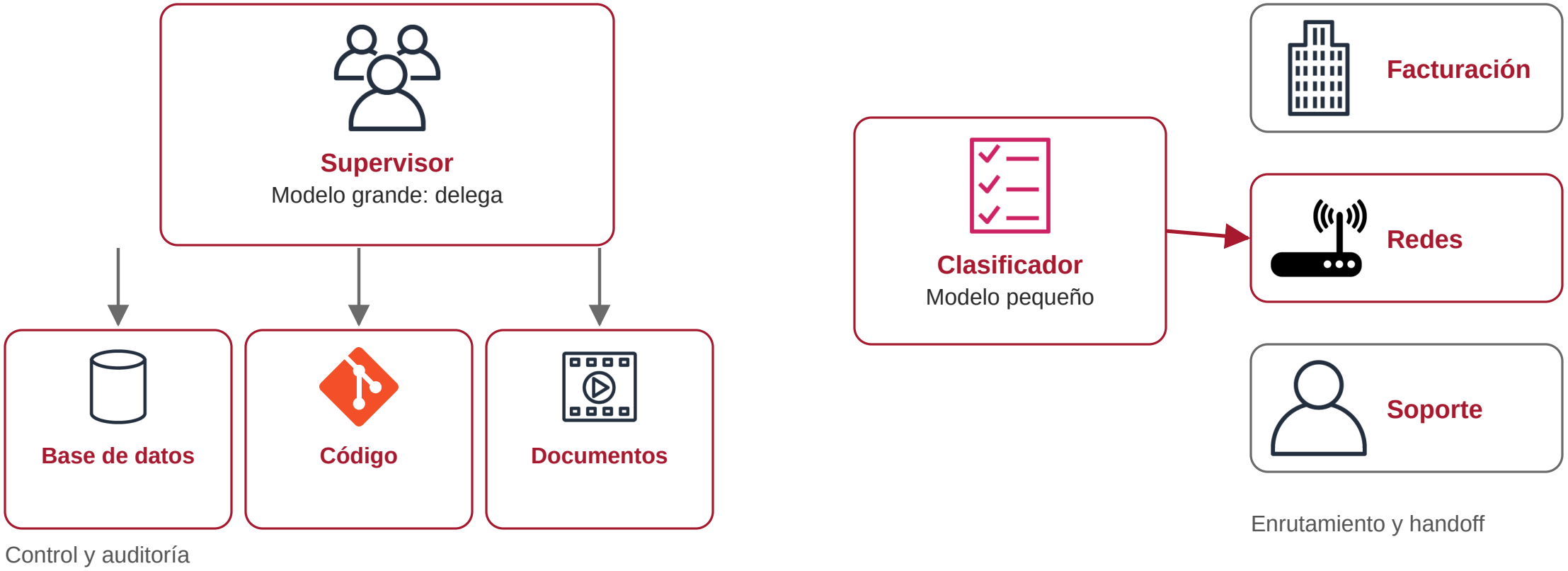
Defensa en profundidad antes y después

PII = información personal identificable

Fuente: Amazon Bedrock Guardrails (docs.aws.amazon.com/bedrock); criterio de diseño del curso

Patrones de orquestación multi-agente

Supervisor central frente a enrutamiento por especialidad

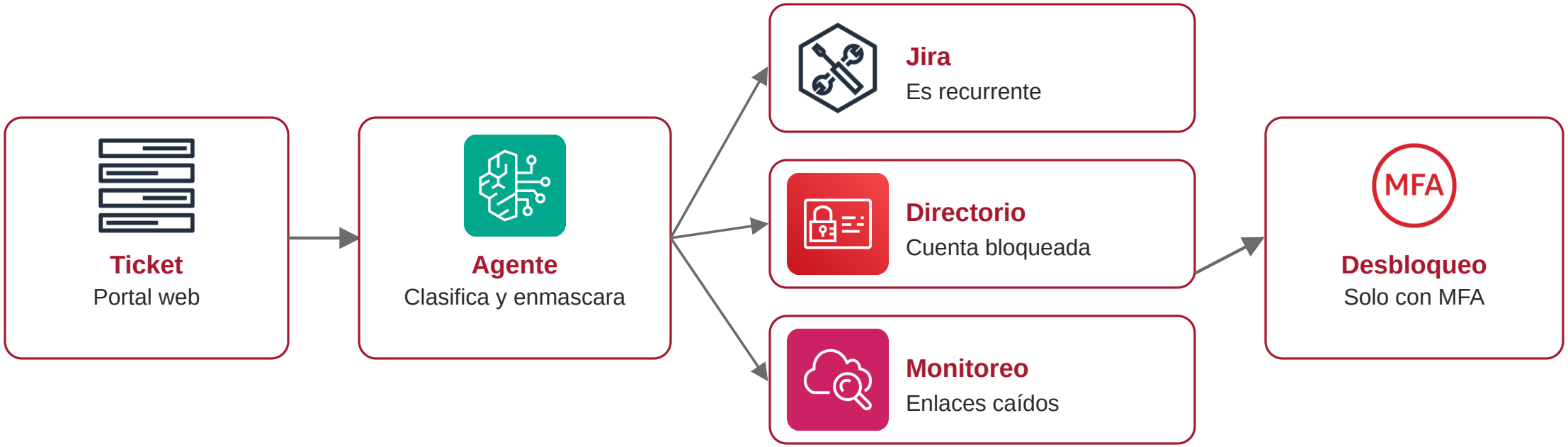


Fuente: criterio de diseño del curso (supervisor y handoff)

CASO ILUSTRATIVO

Mesa de ayuda agéntica con servidores MCP

Caso ilustrativo, no es una empresa real: las herramientas cambian



El agente no cambia

MFA = autenticación de múltiples factores · MCP = protocolo de contexto de modelos

Caso ilustrativo de elaboración propia; MCP (modelcontextprotocol.io)

SÍNTESIS DE LA PARTE 3

Principios de sistemas agénticos

Cuatro reglas para diseñar agentes confiables en la nube

1



ReAct y JSON Schema

Contratos tipados

2



MCP

Recursos, tools y prompts

3



Memoria por capas

Trabajo, episódica, semántica

4



Mínimo privilegio

Guardrails

JSON = notación de objetos de JavaScript · MCP = protocolo de contexto de modelos

Fuente: síntesis de las diapositivas 32 a 44 de esta parte

SEMANA 16 — INTELIGENCIA ARTIFICIAL EN LA NUBE III

Repaso Integral de la Unidad 3

Parte 4 de 4 — Síntesis de Nubes Privadas, Cómputo GPU, Inferencia y Agentes

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) — Ingeniería de Sistemas

Semana 16 — Periodo 2026-2 — Barranquilla, Colombia

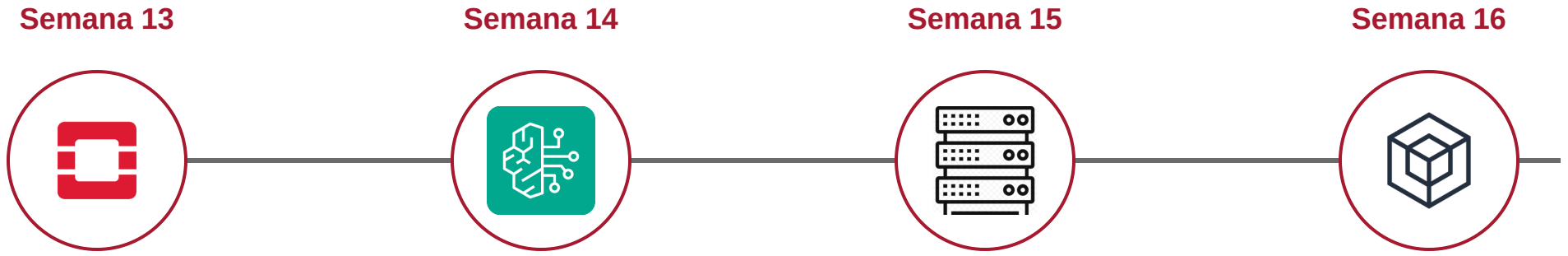
Parte 4 de 4 — Síntesis de Nubes Privadas, Cómputo GPU, Inferencia y Agentes

III = tercera (número romano) · GPU = unidad de procesamiento gráfico

VISIÓN HOLÍSTICA

Mapa de ruta: de la infraestructura base a la IA

Cuatro semanas: nube privada, tokens, aceleración y agentes



Del plano de control al agente

GPU = unidad de procesamiento gráfico · MCP = protocolo de contexto de modelos · IA = inteligencia artificial

Fuente: síntesis de las semanas 13 a 16 de la Unidad 3

Ciclo de vida de una VM en OpenStack

Keystone, Nova, Neutron, Cinder y Glance trabajan en secuencia



Keystone
Token Fernet

Nova API
Cuotas y BD

Neutron
Puerto e IP

Cinder
Volumen

Glance
Imagen qcow2



Nova Compute
Libvirt y KVM: estado ACTIVE

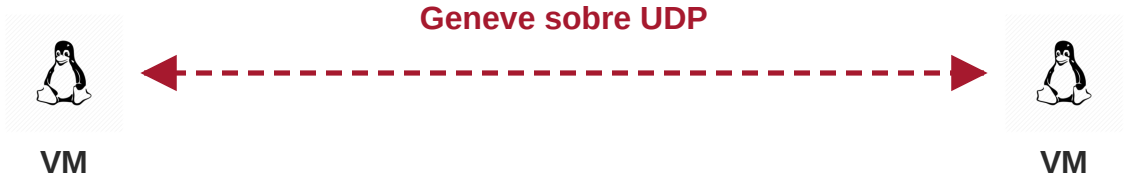
API = interfaz de programación · BD = base de datos · IP = protocolo de internet · KVM = máquina virtual basada en el kernel · VM = máquina virtual

Fuente: documentación de OpenStack, creación de instancias (docs.openstack.org/nova)

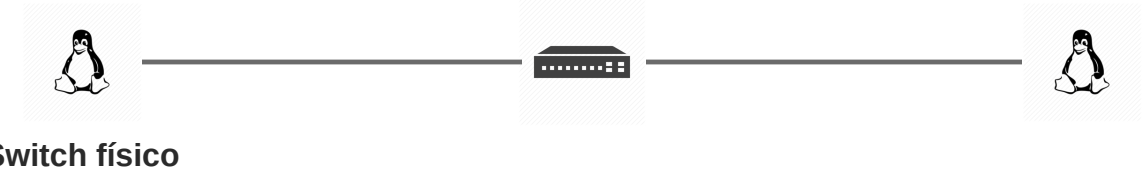
Redes de proyecto (overlay) frente a red de proveedor

Geneve aísla inquilinos; VLAN conecta directo con la red física

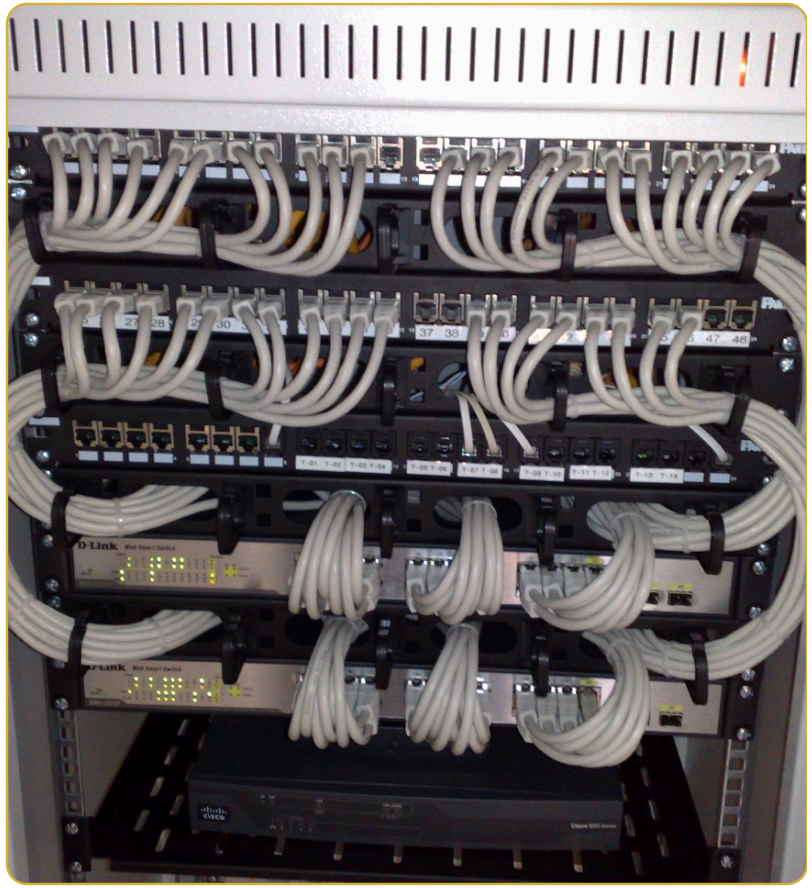
Red de proyecto (overlay)



Red de proveedor (VLAN)



Solo administradores crean estas redes

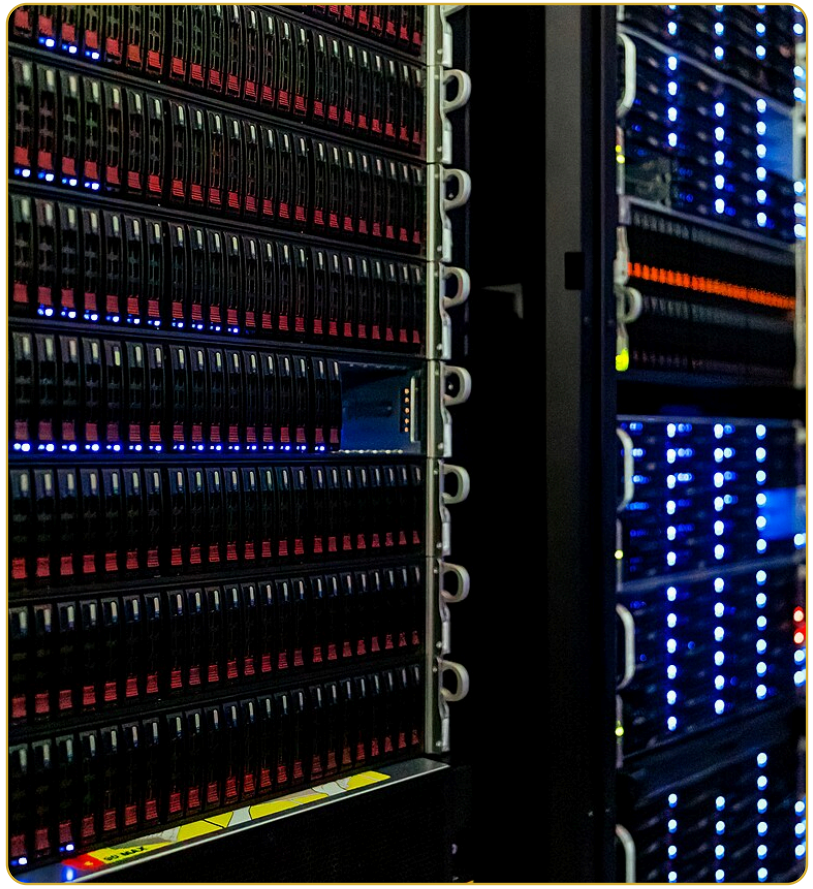


UDP = protocolo de datagramas · VM = máquina virtual

Fuente: OpenStack Neutron, tipos de red (docs.openstack.org/neutron) · Foto: Dsimic, Wikimedia Commons (CC BY-SA 4.0)

Alta disponibilidad: Galera, RabbitMQ y STONITH

Tres nodos de control: dos siempre forman mayoría



Galera
Quórum de 3 nodos de control

RabbitMQ
Quorum queues con Raft

Pacemaker + STONITH
Aísla el nodo que falla

Dos nodos siempre forman mayoría

STONITH = apagado del otro nodo para evitar dos líderes

Fuente: Galera Cluster (galeracluster.com); RabbitMQ quorum queues (rabbitmq.com) · Foto: NOIRLab/NSF/AURA, CC BY 4.0

GPU y memoria HBM3 frente a CPU

El cuello de botella de la inferencia es el ancho de banda

CPU · DDR5

Ancho de banda bajo

GPU · HBM3

3,35 TB/s

NVLink 4

900 GB/s entre GPU

Tensor Cores

Cada token lee los pesos



CPU = unidad central de procesamiento · DDR5 = memoria de sistema de quinta generación · GPU = unidad de procesamiento gráfico · HBM3 = memoria de alto ancho de banda, 3.ª generación · TB = terabyte · GB = gigabyte

Fuente: NVIDIA, H100 Tensor Core GPU (nvidia.com/en-us/data-center/h100) · Foto: Geekerwan, Wikimedia Commons (CC BY 3.0)

Cuantización: AWQ frente a GPTQ

Cuantizar a 4 bits protegiendo el 1 % de pesos sensibles

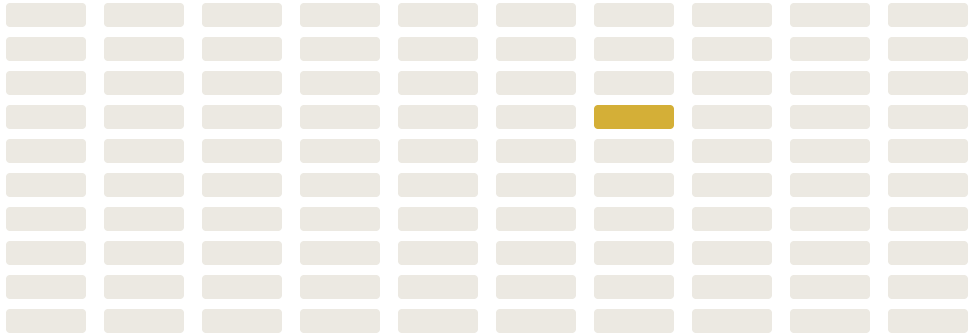
GPTQ
Compensa capa por capa

Lotes pequeños

Optimizado en vLLM

AWQ
Protege lo saliente

Pesos a 4 bits (INT4)



1 % de pesos sensibles

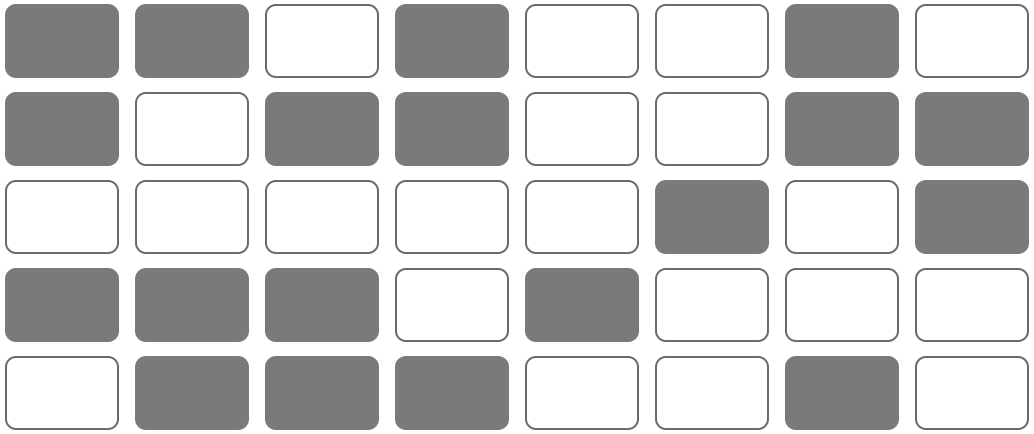
GPTQ = cuantización por error residual por capas · AWQ = cuantización consciente de activaciones · vLLM = motor de inferencia de alto rendimiento

Fuente: Lin et al. (2024), AWQ, MLSys 2024, arXiv:2306.00978; Frantar et al. (2023), GPTQ, arXiv:2210.17323

vLLM: PagedAttention y batching continuo

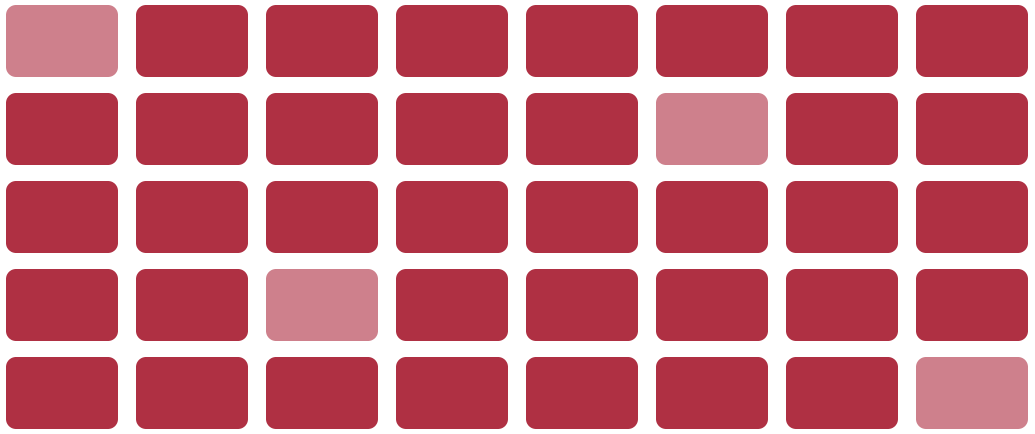
Memoria por páginas y nuevas peticiones en cada token

Reserva contigua



Desperdicio alto por reserva

Páginas de 16 tokens



Casi cero desperdicio

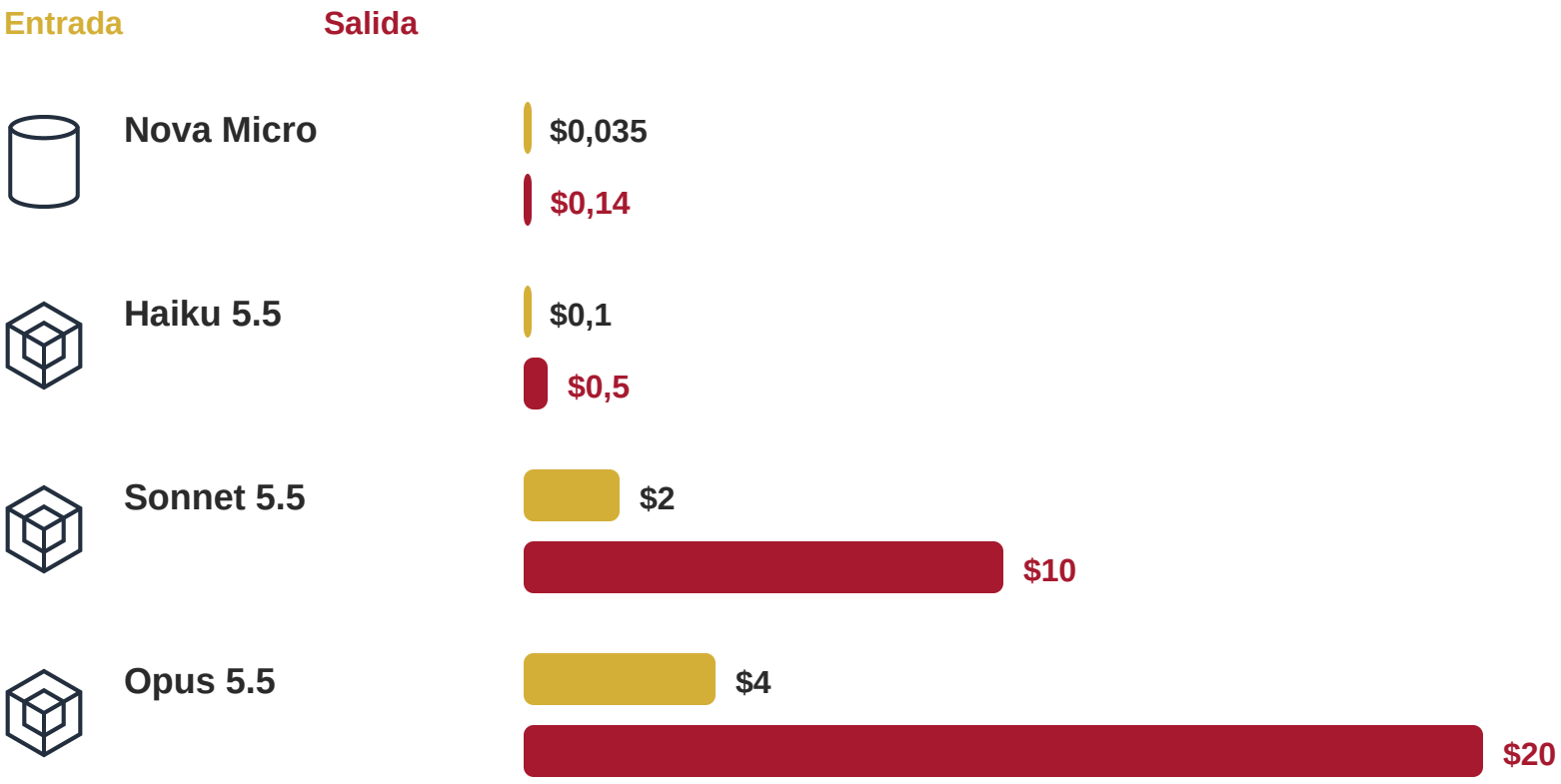
Batching continuo: 2 a 4 veces más rendimiento

vLLM = motor de inferencia de alto rendimiento

Fuente: Kwon et al. (2023), PagedAttention, SOSP 2023, arXiv:2309.06180 (abstract: 2-4× de rendimiento)

Amazon Bedrock: catálogo multimodelo

Tarifas por millón de tokens, región US-East (noviembre 2026)

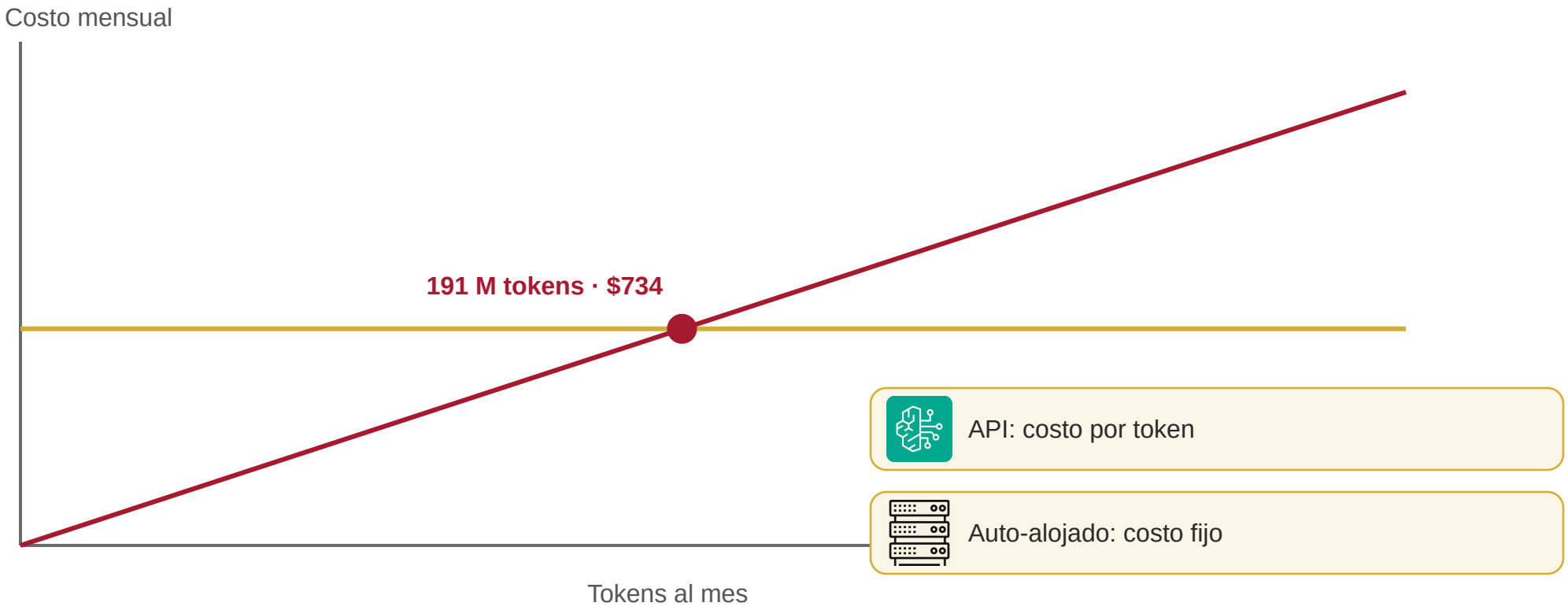


IA = inteligencia artificial · US = Estados Unidos

Fuente: AWS, precios de Amazon Bedrock us-east-1 (aws.amazon.com/bedrock/pricing), consulta 2026-11-05

TCO: ¿auto-alojar o API?

Punto de equilibrio: 191 millones de tokens al mes



API = interfaz de programación · TCO = costo total de propiedad

Fuente: cálculo propio: $734 = $1,006/h × 730 h (g5.xlarge); $0,005 por petición de 1 300 tokens con Sonnet 5.5

Soberanía de datos: Ley 1581 de 2012

Requisitos para tratar datos personales en la nube en Colombia



Finalidad

Autorización del titular



Transferencia

País adecuado o DPA



Seguridad

Cifrado en tránsito y reposo



No entrenar modelos públicos con datos personales

DPA = acuerdo de procesamiento de datos

Fuente: Ley Estatutaria 1581 de 2012 (Colombia); Superintendencia de Industria y Comercio (sic.gov.co)

GUÍA DE DECISIÓN

Matriz de selección: ¿qué tecnología elegir?

Latencia, presupuesto, volumen de datos y criticidad

Soberanía de datos	→ OpenStack local
Velocidad al mercado	→ Amazon Bedrock
Volumen masivo	→ vLLM con AWQ
Agentes seguros	→ MCP con sandbox

vLLM = motor de inferencia de alto rendimiento · AWQ = cuantización consciente de activaciones · MCP = protocolo de contexto de modelos

Fuente: síntesis de las diapositivas 48 a 56 de esta parte

PERFIL PROFESIONAL

El arquitecto de soluciones en la era de la IA

De aprovisionar VM a diseñar sistemas cognitivos confiables

Arquitecto clásico

- Redes y VPC
- Balanceadores
- Discos

Arquitecto de IA

- **Flujos de inferencia**
- **Guardrails y seguridad**
- **Costo y caché**

Ya no es caja negra

VPC = red virtual privada · IA = inteligencia artificial · VM = máquina virtual

CHECKLIST DE ENTREGA

Listos para producción

Seis preguntas antes de desplegar un sistema de IA en la nube

1 Radio de impacto acotado

2 Sandboxes aislados

3 Presupuesto y cuotas

4 Guardrails activos

5 Ley 1581 cumplida

6 TCO validado

TCO = costo total de propiedad · IA = inteligencia artificial

Fuente: síntesis del curso; Ley 1581 de 2012 (sic.gov.co)

CIERRE DE CURSO

Conclusión de la Unidad 3: el futuro de la nube

Infraestructura, eficiencia y gobierno, integrados

Fundamentos

- Galera
- Redes OVN

Eficiencia

- AWQ y vLLM
- Serverless

Gobierno

- MCP
- Firecracker

IA confiable al servicio de la sociedad

OVN = red virtual abierta · AWQ = cuantización consciente de activaciones · vLLM = motor de inferencia de alto rendimiento · MCP = protocolo de contexto de modelos · IA = inteligencia artificial